

Architecture Microservices & Cloud Native

Plateforme de Communication Temps Réel (Style Discord)

Stack Technique Professionnelle :

ASP.NET Core · gRPC · PostgreSQL · Angular · Kubernetes

Objectif du Projet

Développer une application distribuée scalable, utilisant une base de données relationnelle robuste (PostgreSQL), un frontend réactif d'entreprise (Angular) et une orchestration conteneurisée avancée (Kubernetes).

Table des matières

1	Présentation Technique	2
1.1	Contexte	2
1.2	Pourquoi ce choix technologique ?	2
2	Architecture Système	2
2.1	Cartographie des Services	2
3	Flux de Données (Workflow)	2
4	Guide de Réalisation : Roadmap	3
4.1	Phase 1 : Socle et Contrats (.proto)	3
4.2	Phase 2 : Persistance avec PostgreSQL	3
4.3	Phase 3 : Le Backend (Services .NET)	3
4.4	Phase 4 : Frontend Angular (L'Expertise RxJS)	3
4.5	Phase 5 : Conteneurisation (Docker)	3
4.6	Phase 6 : Orchestration Kubernetes (Le niveau Pro)	4
5	Détails Techniques Importants	4
5.1	Gestion de la Configuration	4
5.2	Communication Inter-Pod	4
6	Conclusion et Critères de Succès	4

1 Présentation Technique

1.1 Contexte

Le projet consiste à bâtir une alternative légère aux plateformes comme Slack ou Discord. Au-delà des fonctionnalités métier, l'objectif pédagogique est de maîtriser la communication inter-services (gRPC), la persistance distribuée (PostgreSQL) et le déploiement cloud-native (Kubernetes).

1.2 Pourquoi ce choix technologique ?

- **PostgreSQL** : Choisi pour sa robustesse, sa gestion avancée du JSONB (utile pour des logs ou métadonnées de messages) et sa conformité ACID stricte.
- **Angular** : Préféré ici à React pour son intégration native de **RxJS**. Les flux de données gRPC (streaming) correspondent parfaitement au pattern *Observable* d'Angular.
- **Kubernetes (K8s)** : Permet de dépasser les limites de Docker Compose en gérant l'auto-scaling, la résilience (self-healing) et les mises à jour sans interruption.

2 Architecture Système

L'architecture suit le modèle *Database per Service*. Changer la base de données vers PostgreSQL ne modifie pas la structure des microservices, mais impacte la configuration de l'infrastructure (fichiers YAML K8s et Connection Strings).

2.1 Cartographie des Services

1. Ingress Controller / API Gateway

- *K8s Object* : Ingress (Nginx ou Traefik).
- *Rôle* : Point d'entrée unique du cluster. Gère le routage HTTP/2 nécessaire pour gRPC-Web.

2. Auth Service

- *Stack* : .NET 8 Web API.
- *Persistence* : **PostgreSQL (Database : AuthDB)**.
- *Rôle* : Gestion des identités, hachage des mots de passe, génération JWT.

3. Business Service

- *Stack* : .NET 8 Web API + Client gRPC.
- *Persistence* : **PostgreSQL (Database : BusinessDB)**.
- *Rôle* : Logique métier (Salons, Historique des messages).

4. Notification Service (Le Hub Temps Réel)

- *Stack* : .NET 8 gRPC Server.
- *Persistence* : Redis (Optionnel, pour le cache des sessions) ou In-Memory.
- *Rôle* : Streaming bidirectionnel vers les clients Angular.

3 Flux de Données (Workflow)

Le flux respecte le principe CQRS simplifié (Command Query Responsibility Segregation) :

1. **Écriture (Command)** : L'utilisateur envoie un message via Angular.

Angular **HTTPPOST** *Gateway* *BusinessService* **EFCore** *PostgreSQL*

2. **Propagation (Event)** : Une fois persisté, le Business Service notifie le système de streaming.

BusinessServicegRPCInterneNotificationService

3. **Lecture Temps Réel (Stream)** : Le Notification Service pousse la donnée dans le tuyau ouvert.

NotificationServicegRPC – WebStreamAngular(RxJSObservable)

4 Guide de Réalisation : Roadmap

4.1 Phase 1 : Socle et Contrats (.proto)

Action : Créer la bibliothèque partagée `Shared.Protos`.

- Définir les messages Protobuf.
- **Astuce** : Penser à ajouter l'option `csharp_namespace` dans les fichiers `.proto` pour une génération propre.

4.2 Phase 2 : Persistance avec PostgreSQL

Action : Implémenter la couche Data dans Auth et Business Services.

- Packages NuGet : `Npgsql.EntityFrameworkCore.PostgreSQL`.
- Configuration : Dans `Program.cs`, utiliser `UseNpgsql()`.
- Migrations : Générer les migrations EF Core et s'assurer qu'elles s'appliquent au démarrage du conteneur (via un script ou `context.Database.Migrate()` au startup).

4.3 Phase 3 : Le Backend (Services .NET)

Points d'attention :

- **CORS** : Indispensable pour permettre au Frontend (hébergé ailleurs) de contacter l'API. Autoriser `Grpc-Status`, `Grpc-Message` dans les headers exposés.
- **gRPC-Web** : .NET ne supporte pas gRPC natif dans le navigateur. Il faut activer le middleware `UseGrpcWeb()` sur les endpoints gRPC.

4.4 Phase 4 : Frontend Angular (L'Expertise RxJS)

C'est ici qu'Angular brille par rapport à React pour ce projet.

Structure :

- **Services** : Créer un `ChatService` qui encapsule le client gRPC généré.
- **Protoc** : Utiliser `protoc` avec le plugin `ts-protoc-gen` pour générer les services Angular typés.
- **Logique Réactive** :
 - Créer un `Subject` ou `BehaviorSubject` pour stocker la liste des messages.
 - La fonction de stream gRPC doit appeler `subject.next(newMessage)` à chaque réception.
 - Dans le composant HTML : utiliser le pipe `async (*ngFor="let msg of messages|async")pouraffi`

4.5 Phase 5 : Conteneurisation (Docker)

Créer un `Dockerfile` optimisé pour chaque service (Build en multi-stage pour réduire la taille des images).

4.6 Phase 6 : Orchestration Kubernetes (Le niveau Pro)

Au lieu d'un `docker-compose`, nous définirons des manifestes K8s (dossier `/k8s`).

Fichiers à créer :

1. **Postgres-StatefulSet.yaml** : Pour la base de données (StatefulSet garantit la persistance des données via un PersistentVolume).
2. **Deployments.yaml** : Pour Auth, Business, Notification et Frontend. Définir les **Replicas** (ex : 2 replicas pour le Business Service pour tester le load balancing).
3. **Services.yaml** : De type **ClusterIP** pour la communication interne.
4. **Ingress.yaml** : Configuration du routage externe (ex : `api.monapp.local` redirige vers les services).

5 Détails Techniques Importants

5.1 Gestion de la Configuration

Les chaînes de connexion PostgreSQL ne doivent pas être en dur.

- **En Dev** : `appsettings.Development.json`.
- **En K8s** : Utiliser des **ConfigMaps** (pour les URL non sensibles) et des **Secrets** (pour les mots de passe DB et clés JWT).

5.2 Communication Inter-Pod

Dans Kubernetes, les services se parlent via leur nom DNS interne. Exemple : Le Business Service contactera le Notification Service via l'URL `http://notification-service:80`, et non via `localhost`.

6 Conclusion et Critères de Succès

Ce projet valide des compétences avancées en Fullstack Engineering. **Critères de réussite :**

L'authentification JWT fonctionne et protège les routes.

Les données sont persistées dans PostgreSQL.

Angular reçoit les messages en temps réel sans rafraîchissement (via gRPC-Web).

Le tout est déployable via `kubectl apply -f k8s/`.